Rīgas 64. vidusskola

Tiešas un JIT balstītas interpretācijas salīdzinājums CHIP-8 emulatorā

Zinātniski pētniecības darbs datorzinātņu un informātikas nozarē

Darba autors:
Rīgas 64. vidusskolas 12. INZ klases skolnieks Ričards Lūciņš
Darba vadītājs:
Rīgas 64. vidusskolas programmēšanas skolotājs Edvards Bukovskis

Rīga, 2025

Anotācija

Zinātniski pētniecisko darbu “Tiešas un JIT balstītas interpretācijas salīdzinājums CHIP-8 emulatorā” izstrādāja Rīgas 64. vidusskolas 12. INZ klases skolnieks Ričards Lūciņš, darba vadītājs – Rīgas 64. vidusskolas programmēšanas skolotājs Edvards Bukovskis.

Teorētiskajā daļā tika izpētīti emulatoru darbības pamatprincipi un fiziskās aparatūras procesu modelēšana programmatūras slānī. Tika apskatīta tiešā interpretācija un JIT *(just-in-time)* balstītā interpretācija, kā arī veikta šo pieeju teorētiskā salīdzināšana, analizējot to iespējamo ietekmi uz sistēmas veiktspēju.

Praktiskajā daļā autors izstrādāja divas CHIP-8 emulatora versijas, realizējot dažādas interpretācijas metodes. Balstoties uz veiktajiem mērījumiem kontrolētā testa vidē, tika veikta iegūto datu analīze, salīdzinot emulatoru veiktspēju īstermiņa un ilgtermiņa operācijās.

Atslēgvārdi: emulators, CHIP-8, aparatūras abstrakcija, tiešā interpretācija, JIT kompilācija, dinamiskā koda ģenerēšana, izpildlaika optimizācija.

Abstract

The scientific research paper "Comparison of direct and JIT-based interpretation in the CHIP-8 emulator" was developed by Ričards Lūciņš, a 12th grade student at Riga Secondary School No. 64, under the supervision of Edvards Bukovskis, a programming teacher at Riga Secondary School No. 64..

The theoretical part examined the basic principles of emulator operation and the modeling of physical hardware processes in the software layer. Direct interpretation and JIT *(just-in-time)* based interpretation methods were examined, and a theoretical comparison of these approaches was carried out, analyzing their possible impact on system performance.

In the practical part, the author developed two versions of the CHIP-8 emulator, implementing different interpretation methods. Based on measurements taken in a controlled test environment, the data obtained was analyzed, comparing the performance of the emulators in short-term and long-term operations.

Keywords: emulator, CHIP-8, hardware abstraction, direct interpretation, JIT compilation, dynamic code generation, runtime optimization.

Saturs

Anotācija 2
Abstract 2
Saturs 3
Ievads 4
1. Teorētiskā daļa 5
1.1. Emulatoru darbības principi 5
1.1.1. Fiziskās aparatūras procesu modelēšana programmatūras slānī 5
1.2. Instrukcijas interpretācijas pieejas 6
1.2.1. Tieša interpretācija 6
1.2.2. JIT balstīta interpretācija 7
1.3. CHIP-8 kā pētījuma platforma 8
2. Praktiskā daļa 9
2.1. Emulatoru izstrādes process 9
2.2. Testa vides apraksts 10
2.3. Rezultātu analīze 10
2.3.1. Īstermiņa veiktspējas analīze 10
2.3.2. Ilgtermiņa veiktspējas analīze 12
2.3.3. Rezultātu interpretācija 14
2.4. Testa vides ierobežojumi un kļūdas 15
Secinājumi 15
Izmantotās literatūras saraksts 17
Pielikums 17

Ievads

Mūsdienu informācijas tehnoloģiju (IT) attīstība ir cieši saistīta ar nepieciešamību palaist kodu dažādās platformās un arhitektūrās. Šādos gadījumos tiek izmantoti emulatori, kuru galvenā funkcija ir palaist programmu(-as) uz arhitektūras vai sistēmas, kurai tā sākotnēji nav bijusi paredzēta, izmantojot mašīnkoda pārtulkošanu un interpretēšanu. Emulatorus var izmantot dažādās situācijās no vecāku paaudžu datoru un datorspēļu sistēmu saglabāšanas līdz pat programmatūras testēšanai dažādās vidēs un koda portēšanai uz citām platformām. Emulatoru darbības pamatā ir *fetch-decode-execute* (FDE) cikls, kur instrukcijas tiek nolasītas, analizētas un izpildītas secīgi. Vienkāršākajos emulatoros šis modelis tiek īstenots tiešajā interpretācijā, kur tiek izmantots augsta līmeņa abstrakcijas slānis, emulators nevis ģenerē mašīnkodu, bet gan realizē emulētās platformas loģiku, izmantojot saimniekvalodas iebūvētas konstrukcijas. JIT *(just-in-time)* balstīta interpretācija balstās uz to pašu FDE modeli, bet to papildina ar instrukciju dinamisku pārtulkošanu mērķa platformas mašīnkodā. Šādā gadījumā izpilde vairs nenozīmē tiešu instrukcijas interpretāciju emulatora līmenī, bet gan ģenerēta mašīnkoda izpildi, kas veic instrukciju noteikto loģiku.

Šajā darbā tiks salīdzinātas abas iepriekšminētās interpretācijas metodes, un apskatītas atšķirības efektivitātē un veiktspējā ar CHIP-8 emulatora palīdzību.

Darba mērķis: salīdzināt tiešās un JIT balstītās interpretācijas metodes, analizēt pārtulkošanas un izpildes laiku, veiktspēju, efektivitāti, izmantotos resursus, secināt vājās un stiprās puses katrai no metodēm, ja tādas ir.

Hipotēze: izmantojot JIT balstītu interpretāciju, izpildes laiks būs īsāks, taču, izmantojot tiešo interpretāciju, tiks patērēti mazāk sistēmas resursi.

Uzdevumi:

1. Izveidot divus CHIP-8 emulatorus, vienu, kurš izmanto tiešo interpretācijas pieeju, un otru, kurš izmanto JIT balstītas interpretācijas metodi;
2. Izveidot kritērijus, pēc kuriem tiks veikta analīze un salīdzināšana;
3. Veikt testus uz emulatoriem;
4. Ievākt datus par izmantoto sistēmas resursu daudzumu, izpildes laiku un veiktspēju;
5. Izpētīt, kura metode ir visefektīvākā;
6. Veikt datu analīzi un izvirzīt secinājumus.

Darbā izmantotās metodes:

1. Literatūras analīze;
2. CHIP-8 emulatoru praktiskā izstrāde;
3. Emulatoru eksperimentālā testēšana;
4. Datu ievākšana un analīze.

Darba struktūra: darbs sastāv no ievada, 2 nodaļām, 13 apakšnodaļām, secinājumiem, izmantoto informācijas avotu saraksta un pielikuma. Darbā ir 6 attēli un 1 tabula.

1. Teorētiskā daļa

1.1. Emulatoru darbības principi

Emulācija ir process, kurā viena datorsistēma (turpmāk - saimniekdators) reproducē vai atdarina citas sistēmas (turpmāk - emulējamās sistēmas) uzvedību vai tās funkcionalitāti. Emulators ir programmatūras un/vai aparatūras (arī tās daļas(-u)) kopums, kas nodrošina emulāciju [6], tādējādi ļaujot saimniekdatora sistēmai darbināt programmatūru, kas sākotnēji bija paredzēta emulējamai sistēmai, precīzi atdarinot tās procesora, atmiņas un perifērijas darbību. Atšķirībā no simulācijas, kas var atdarināt tikai emulējamās sistēmas ārējo uzvedību, emulācija tiecas pēc iespējas precīzāk replicēt tās iekšējo stāvokli un loģiku/algoritmus. [5]

Emulatora pamatuzdevums ir radīt virtuālu vidi, kas tulko emulējamās sistēmas instrukcijas uz tādu formu, kādu saprot saimniekdators. Lai to nodrošinātu, emulatoram ir jāreplicē vairāku komponentu darbība:

Centrālais procesors (CPU). Šis ir emulatora kodols un bieži vien vissarežģītākā komponente, kuru vajag implementēt. Sakarā ar procesoru, emulatoriem ir tādi uzdevumi kā reģistru (dati, kas atrodas pašā procesorā) un karogu uzturēšana, kuri tiek modificēti, atkarībā no kādas operācijas rezultāta (piemēram, vai kādas operācijas rezultāts ir nulle, negatīvs, vai notikusi pārnese). Emulatoram ir arī jāspēj sekot līdzi programmas skaitītājam (angliski: *program counter* jeb PC), kā arī to modificēt, ja nepieciešams, jo programmas skaitītājs ir reģistrs, kas norāda uz nākamās izpildāmās instrukcijas adresi atmiņā [2]. Procesora emulācija bieži vien prasa precīzu katras instrukcijas loģikas/algoritma realizāciju.

Atmiņas apakšsistēma. Emulatoram ir jāpiešķir daļa no saimniekdatora operatīvās atmiņas (angliski: *Random Access Memory* jeb RAM), lai emulētu emulējamās sistēmas atmiņu. Ir būtiski precīzi ievērot tās atmiņas kartējumu (angliski: *memory map)*, nodrošinot, ka emulējamā sistēma piekļūst datiem pareizajās adresēs. [1]

Ievades un izvades (angliski: *Input/*Output jeb I/O) sistēma. Tā kā emulatorā palaistās programmas, it īpaši vecākas arhitektūras, bieži ievada datus tieši video atmiņā vai lasa ievades signālus no specifiskiem aparatūras portiem (piemēram, klaviatūras kontrollera), emulatoram šie signāli ir jāpārtver. Tie ir jāpārveido mūsdienu operētājsistēmai saprotamos formātos – zīmējot pikseļus logā vai nolasot klaviatūras taustiņu nospiedumus un nododot tos atpakaļ virtuālajai videi (angliski: virtual environment). [2]

Vēl viens būtisks aspekts, lai emulators precīzi un veiksmīgi emulētu kādu sistēmu, ir laika sinhronizācija. Bieži vien vecākas sistēmas un arhitektūras darbojas ar noteiktu frekvenci. Šī iemesla dēļ emulatoriem, kas darbojas uz mūsdienu sistēmām, ir bieži vien jāierobežo instrukciju izpildes ātrums, lai tas atbilstu oriģinālās sistēmas veiktspējai, jo pretējā gadījumā emulējamā sistēma var izpildīt kādas noteiktas programmatūras daudz ātrāk, nekā tas bija paredzēts. [2]

1.1.1. Fiziskās aparatūras procesu modelēšana programmatūras slānī

Lai izprastu emulatora darbību, ir būtiski nošķirt fiziskas aparatūras (*bare metal*) procesus no to programmatūras simulācijas. Fiziskā sistēmā instrukciju izpilde ir elektromehānisks process: vadības bloks manipulē ar sprieguma līmeņiem reģistros un operatīvajā atmiņā caur loģiskajiem vārtiem. Emulācijas procesā šī fiziskā realitāte tiek aizstāta ar augsta līmeņa programmatūras abstrakciju [9].

Šajā posmā emulators aizstāj fiziskās procesora arhitektūras specifiku ar algoritmiskiem modeļiem, pārejot uz loģisku operāciju izpildi saimniekvalodas (piemēram, C++, Python vai JavaScript) vidē. Šī transformācija ietver trīs galvenos aspektus:

Sistēmas stāvokļa virtualizācija: Emulējamās platformas fiziskie komponenti tiek reprezentēti kā abstraktas datu struktūras. Procesora reģistri programmatūrā tiek definēti kā mainīgie vai masīva elementi (piemēram, `V[x]` masīvs CHIP-8 gadījumā). Savukārt operatīvā atmiņa tiek modelēta kā nepārtraukts, dinamiski piešķirts atmiņas apgabals jeb buferis saimniekdatora operatīvajā atmiņā, aizstājot fizisko adrešu kopni ar programmatūras indeksāciju [9].

Aparatūras loģikas aizstāšana ar funkcijām: Tā vietā, lai fiziski aktivizētu procesora aritmētiski loģisko bloku (ALU), emulators izmanto programmēšanas valodas iebūvētos operatorus. Piemēram, instrukcija, kas veic vērtības pieskaitīšanu reģistram, tiešajā interpretācijā tiek realizēta kā augsta līmeņa operācija. Šādā veidā aprēķinu loģika tiek izolēta no fiziskās aparatūras specifikas [9].

Aparatūras neatkarība un platformneatkarība: Izmantojot augsta līmeņa valodas funkcijas, tiek panākta izolācija no saimniekdatora (*host*) aparatūras specifiskās instrukciju kopas arhitektūras (ISA). Emulatoram nav nepieciešams ģenerēt specifisku mašīnkodu katram procesora tipam (piemēram, x86 vai ARM); tas paļaujas uz programmēšanas valodas nodrošināto loģisko rezultātu, ko garantē kompilators vai interpretators [8].

Šis abstrakcijas slānis nodrošina augstu koda pārnesamību un vienkāršo izstrādes procesu, taču tas ir galvenais cēlonis veiktspējas zudumiem, jo katra emulētā operācija prasa vairākus saimniekdatora procesora ciklus, lai to apstrādātu programmatūras līmenī [8].

1.2. Instrukcijas interpretācijas pieejas

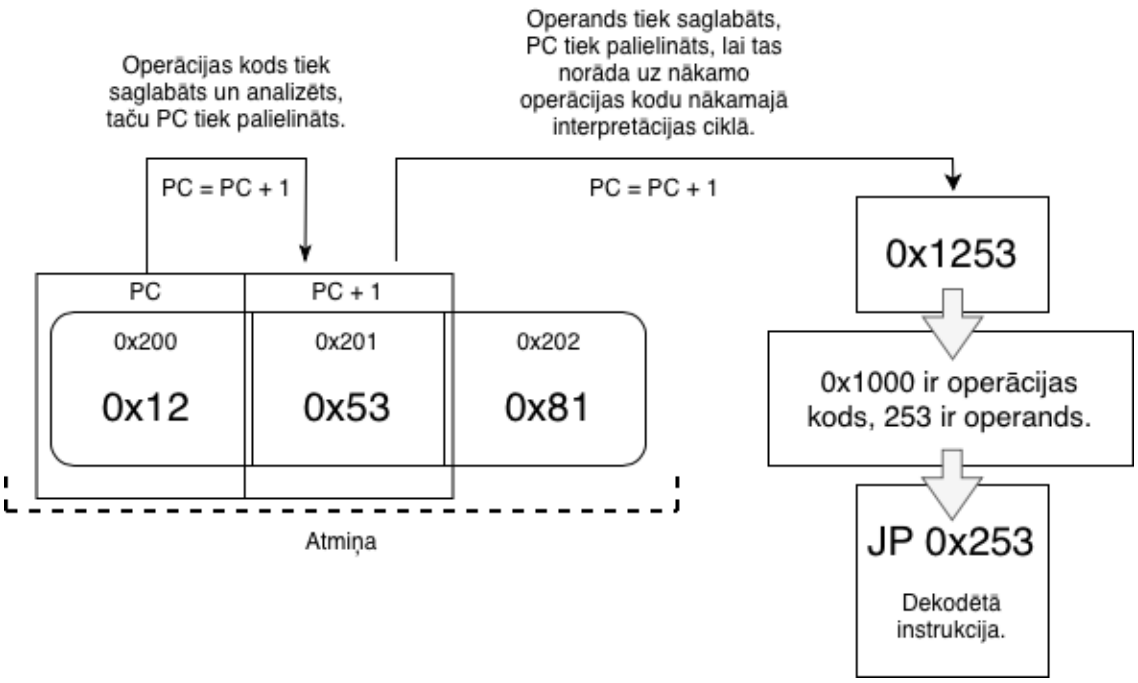
1.2.1. Tieša interpretācija

Šī pieeja sevī ietver katras instrukcijas iegūšanas, pārtulkošanas un izpildīšanas ciklu, kuru centrālais procesors nepārtraukti atkārto sākot no datora ieslēgšanas līdz pat izslēgšanai, caur augsta līmeņa abstrakcijas slāni. Šis cikls ir pamats visām mūsdienu datoru darbībām. [3]

Fetch (iegūšanas) solis: no atmiņas tiek iegūta nākamā izpildāmā instrukcija, uz kuru norāda programmas skaitītājs. Iegūtā instrukcija tiek saglabāta bieži vien atsevišķā reģistrā, kuru sauc par "pašreizējās instrukcijas reģistru" (angliski: Current Instruction Register jeb CIR). Pēc instrukcijas izpildes programmas skaitītājs tiek palielināts, lai norādītu uz nākamo instrukciju, un tā būtu izpildīta nākamajā ciklā. [3]

Decode (dekodēšanas/pārtulkošanas) solis: instrukcija tiek iegūta no CIR un interpretēta, lai noteiktu, kāda operācija ir jāveic (piemēram, saskaitīšana vai atmiņas pārvaldīšana), lai to izdarītu instrukcija tiek sadalīta operācijas kodā (angliski: operation code jeb opcode), lai zinātu, kas datoram ir jādara, un operandā (angliski: operand), lai iegūtu instrukcijas argumentus, kuri var būt, piemēram, skaitļi vai adreses atmiņā, kas nosaka, kam vai ar ko šī operācija ir jāveic. Ja emulējamā sistēma būtu fiziska aparatūra, kas nedarbotos virtuālajā vidē, tad šajā mirklī vadības bloks nosūtītu attiecīgos signālus procesora komponentēm, piemēram aritmētiski loģiskajai iekārtai (ALU), lai sagatavotos instrukcijas izpildei. [3]

Execute (izpildes) solis: šajā solī instrukcija, kuru pieprasa operācijas kods, tiek izpildīta. Emulators tiešajā interpretācijā izmantos augsta līmeņa abstrakcijas slāni un saimniekvalodas piedāvātās konstrukcijas, lai izpildītu šo instrukciju, kā rezultātā dati var būt saglabāti vai nolasīti no atmiņas reģistriem, kā arī programmas skaitītājs var tikt mainīts, ja to paredzēja operācijas kods vai sākās atzarojums. [3]



1.1. attēls: FDE cikla modelis)

Lai gan šī interpretācijas pieeja ir precīza (ar šo metodi var precīzi un perfekti atdarināt emulējamās sistēmas darbības un uzvedību), tā varētu būt lēna, sakarā ar tās modeli un trūkumiem:

Nepārtraukta cikla uzturēšana: katru reizi, kad cikls ir pabeigts, ir jāveido jauns, kas nozīmē nepārtrauktu datu iegūšanu, analīzi un izpildi pat tad, ja iepriekš izpildītā instrukcija ir identiska nākamajai.

Nepietiekami efektīva reģistru un atmiņas pārvalde: emulatori ar augsta līmeņa abstrakcijas slāni bieži vien atdarina virtuālās vides reģistrus izmantojot saimniekdatora operatīvo atmiņu, tā vietā, lai lietotu saimniekdatora procesora pieejamos reģistrus.

Nepastāv atzarojumu prognozēšana: mūsdienu procesori lieto zaru prognozēšanu, lai paātrinātu izpildi. Emulators ar tiešu interpretāciju neizmanto šīs iespējas, jo pirms instrukcijas izpildes vajag pilnībā pabeigt iepriekšējo ciklu.

1.2.2. JIT balstīta interpretācija

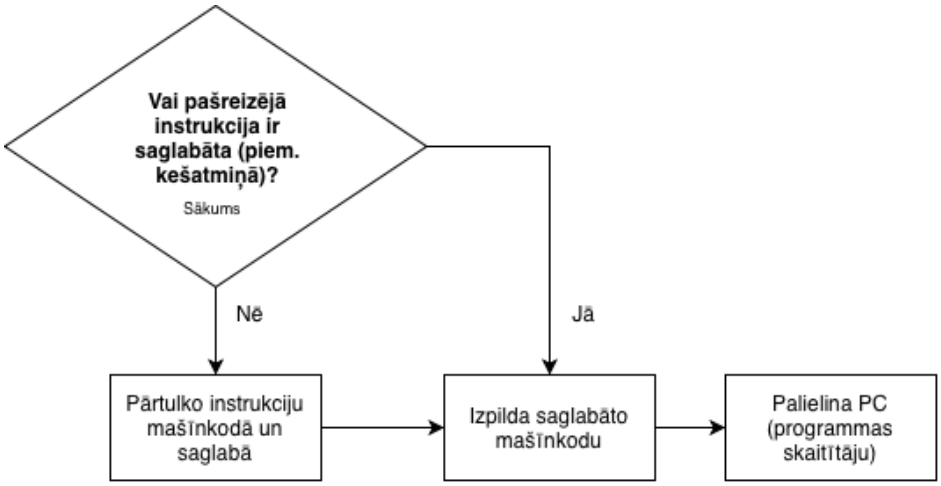
JIT balstīta interpretācija ir uzlabota koda izpildes un interpretācijas metode, kas apvieno tiešās interpretācijas un kompilācijas (koda pārvēršana mērķa platformas atbalstītā mašīnkodā pirms izpildes) priekšrocības. [3] Šī tehnoloģija pirmo reizi parādījās 1980. gados, taču plašu popularitāti ieguva kopā ar tādu virtuālu mašīnu kā Java Virtual Machine (JVM) un .NET Common Language Runtime (CLR) attīstību. [3]

Atšķirībā no tiešās interpretācijas, kur katra instrukcija ir atkārtoti iegūta un pārtulkota neatkarīgi ne no kā, JIT balstīts interpretators:

- **Analizē kodu izpildes laikā:** kamēr interpretators darbojas, JIT identificē tās koda daļas, kuras tiek izpildītas visvairāk.
- **Kompilē kodu tieši:** kad kāds koda bloks ir izpildīts pietiekami daudz reižu, JIT to kompilē saimniekdatora platformas mašīnkodā. [4]
- **Saglabā un izpilda:** pārtulkoto mašīnkodu saglabā atmiņā. Nākamajā reizē, kad programma sasniedz šo pašu koda bloku, JIT izpilda jau gatavo mašīnkodu, apejot lēno tiešās interpretācijas pieeju.

JIT balstītas interpretācijas galvenie ieguvumi implementācijas dizaina dēļ ir labāka veiktspēja, jo kompilēts mašīnkods darbojas daudz ātrāk nekā interpretēts kods, tādējādi ievērojami paātrinot bieži izpildītu koda bloku darbību, kā arī JIT dod iespēju optimizēt kodu, jo JIT kompilators var veikt optimizācijas, kas nav iespējamas tiešā interpretācijā, piemēram, reģistru piešķiršanu no saimniekdatora procesora, tādejādi ģenerējot efektīvāku mašīnkodu nekā statiski kompilēts kods [4], taču JIT interpretācijai savas pieejas dizaina dēļ varētu būt sagaidāmi sekojoši trūkumi:

- **Pārtulkošanas slodze programmas palaišanas brīdī:** sākotnēji JIT balstīts interpretators veic koda analīzi un pārtulkošanu, kas var palēināt programmas palaišanas laiku salīdzinājumā ar tiešo interpretāciju. Ja koda bloks, kuru JIT pārtulkoja mašīnkodā, tiek izpildīts tikai vienu reizi, JIT ieguldītais laiks netiek atpelnīts.
- **Papildu atmiņas noslodze:** JIT prasa papildu operatīvo atmiņu, lai uzglabātu uzģenerēto mašīnkodu.
- **Sarežģītība:** JIT balstīta interpretatora realizācija ir daudz sarežģītāka nekā tieša interpretatora un augsta līmeņa abstrakcijas slāņa realizācija [3].



1.2. attēls: JIT balstītas interpretācijas modelis)

1.3. CHIP-8 kā pētījuma platforma

CHIP-8 ir veca interpretējama programmēšanas valoda, kas tika izstrādāta 1970. Gadu vidū, lai atvieglotu videospēļu veidošanu uz tolaik jaunajiem mikrodatoriem. Lai gan CHIP-8 nav reāla datoru arhitektūra, tā kalpo par virtuālas mašīnas specifikāciju, kas tiek emulēta mūsdienu sistēmās [1]. CHIP-8 raksturo sekojoša aparatūras specifikācija:

- **Atmiņa:** CHIP-8 izmanto 4 KB (4096 baitus) operatīvās atmiņas. Programmas kods tiek ielādēts, sākot ar adresi 0x200, līdz tai adresei atmiņa ir rezervēta emulatoram, kur visbiežāk saglabā fontu, kuru CHIP-8 izmanto teksta izvadei. [2]
- **Reģistri:** CHIP-8 ir 16 vispārējas nozīmes 8 bitu datu reģistri (apzīmēti V0-VF). Reģistrs VF tiek izmantots kā karogs, piemēram, pārneses rezultāta glabāšanai pēc aritmētiskas instrukcijas izpildes. [2]

- **Instrukciju kopa:** CHIP-8 instrukcijas ir 16 bitu garas. Kopumā CHIP-8 atbalsta aptuveni 35 instrukcijas [1]. Tās ir vienkāršas un ietver tikai pamata operācijas, piemēram, aritmētiskās un loģiskās operācijas, atzarojumus un lēcienus uz dažādām adresēm atmiņā, apakšprogrammu izsaukumus un grafisko izvadi. [1]
- **I/O (ievade/izvade):** tiek emulēts 64x32 pikseļu monohroms displejs (katrs pixelis var būt tikai ieslēgts vai izslēgts) un heksadecimāla tastatūra ar 16 taustiņiem. [2]

CHIP-8, kā iepriekš minēts, raksturo kopumā tikai 35 instrukcijas [1], kā rezultātā ir viegli un ātri izveidot CHIP-8 emulatoru gan ar tiešo interpretāciju, gan ar JIT balstītu interpretāciju, līdz ar to emulatoru sarežģītība un izstrādes laiks samazinās, kā arī instrukciju vienkāršu loģiku dēļ var panākt ātra koda pārtulkošanu. Tas nozīmē, ka JIT kompilatoram nav jārisina sarežģītas mūsdienu arhitektūras optimizācijas problēmas. CHIP-8 arī piemīt parasti īsas programmas, kurās atkārtojas vienas un tās pašas instrukciju secības, piemēram, ekrāna zīmēšana vai tastatūras signālu nolasīšana, un CHIP-8 nav sarežģītu I/O vai operētājsistēmas abstrakciju, kas varētu ietekmēt mērījumus. Šo priekšrocību dēļ veiktspējas mērījumi būs gandrīz pilnībā atkarīgi tikai no izvelētās interpretācijas pieejas, un autors varēs izmērīt tīru veiktspējas atšķirību starp tiešo un JIT balstīto interpretāciju.

2. Praktiskā daļa

2.1. Emulatoru izstrādes process

Šī pētījuma praktiskās daļas mērķis bija izveidot divus CHIP-8 emulatorus, kurus autors apvienoja vienotā programmā, kas ļauj salīdzināt divas dažādas instrukciju izpildes pieejas – tiešo un JIT balstīto interpretāciju. Emulatoram bija jāspēj korekti un precīzi emulēt CHIP-8 sistēmu un arhitektūru, izpildīt korekti un precīzi CHIP-8 balstītas programmas, kā arī veikt precīzus veiktspējas mērījumus kontrolētos apstākļos. Galvenie uzdevumi veidošanas procesā bija pilnas CHIP-8 instrukciju kopas realizācija, grafiskās izvades un I/O sistēmas nodrošināšana un testēšanas režīma realizācija, kura nodrošinātu veiktspējas mērījumu izvadi no emulatora.

CHIP-8 emulators tika veidots C++ valodā, jo tā nodrošina ne tikai zema līmeņa kontroli pār atmiņu, bet arī augstu veiktspēju. C++ ļauj efektīvi apvienot gan augsta līmeņa abstrakcijas un konstrukcijas, gan tiešu piekļuvi procesora resursiem un citām sistēmas komponentēm, kas ir būtiski emulatoru izstrādē. Grafiskās izvades un I/O sistēmas realizācijai tika izmantota SDL2 bibliotēka, jo CHIP-8 izmanto vienkāršu, bet bieži atjauninámu 64x32 pikseļu displeju. AsmJit bibliotēka tika izvēlēta JIT balstītās interpretācijas realizācijai, jo tā nodrošina ērtu un drošu veidu, kā ģenerēt x86-64 mašīnkodu programmas izpildes laikā.

Kā iepriekš minēts, emulators ir realizēts kā viena programma ar modulāru struktūru. Palaižot programmu caur termināli, ir iespēja izmantot argumentus, kuru vidū ir argumenti, kas ļauj ne tikai izvēlēties interpretācijas pieeju (tiešo vai JIT balstīto), bet arī iespējot testēšanas jeb “benchmark” režīmu ar noteiktu laika posmu sekundēs. Izveidotais emulators sastāv no vairākiem moduļiem, kuri mijiedarbojas savā starpā emulācijas laikā: centrālā procesora emulācija, atmiņas pārvaldība, grafiskās izvades modulis, ievades apstrādes modulis, kā arī interpretācijas vadības modulis, kurš ir atbildīgs par saimniekvalodas augsta līmeņa konstrukciju izmantošanu instrukciju algoritmu atdarināšanā tiešajā interpretācijā vai mašīnkoda ģenerēšanu JIT balstītajā interpretācijā. Šāda arhitektūra ļauj viegli paplašināt vai modificēt emulatora funkcionalitāti.

Tiešajā interpretācijā katra CHIP-8 instrukcija tiek apstrādāta secīgi. Šī pieeja balstās uz saimniekvalodas augsta līmeņa abstrakcijas slāni un tās konstrukcijām, kā arī uz iepriekšminēto

FDE ciklu, kas atkārtojas līdz programmas izpildes vai emulācijas beigām. Katra cikla laikā no atmiņas tiek nolasīta instrukcija, tā tiek dekodēta un pārtulkota, izmantojot bitu operācijas, un visbeidzot izpildīta.

JIT balstītajā interpretācijas režīmā CHIP-8 instrukcijas netiek interpretētas katrā izpildes ciklā, tā vietā tiek ģenerēts mašīnkods, izmantojot AsmJit bibliotēku.

2.2. Testa vides apraksts

Testēšana un datu mērīšana tika veikta uz viena datora, lai novērstu aparatūras atšķirību ietekmi uz rezultātiem. Izmantotais dators ir balstīts uz x86-64 arhitektūras, kas ir saderīga ar AsmJit ģenerēto mašīnkodu. Procesors nodrošināja kodolus un aparatūras līmeņa kešatmiņu, kas būtiski ietekmēja JIT balstītās interpretācijas efektivitāti. Operatīvās atmiņas apjoms bija pietiekams, lai testu laikā nerastos atmiņas trūkums un netiktu izmantota virtuālā atmiņa, kas varētu ietekmēt mērījumus. Testēšanas laikā netika veikta procesora frekvences palielināšana vai samazināšana, un visi testi tika veikti ar noklusētajiem enerģijas pārvaldības iestatījumiem.

Emulators tika palaists mūsdienīgā 64 bitu operētājsistēmā, kas nodrošina stabilu procesu pārvaldību un precīzus laika mērījumus. Programma tika kompilēta izmantojot C++ kompilatoru ar ieslēgtam optimizācijām, lai iegūtie rezultāti atspoguļotu reālu veiktspēju. AsmJit un SDL2 bibliotēkas tika izmantotas bez papildu modificēšanas, un emulatora realizācijā tika izmantota AsmJit standarta funckionalitāte mašīnkoda ģenerēšanai. Visu testu laikā tika izmantota viena un tā pati emulatora versija, mainot tikai interpretācijas režīmu, kas nodrošina taisnīgu salīdzinājumu.

Kā jau iepriekš minēts, emulators tika izstrādāts tā, lai visas galvenās konfigurācijas iespējas būtu pieejamas emulatora palaišanas brīdī. Testēšanas režīmā emulators automātiski pielāgo savus iestatījumus, piemēram, atslēdzot izpildes ātruma ierobežojumu.

Testēšanai galvenokārt tika izmantota programma 1dcell, kas izmanto plašu instrukciju klāstu, atkārtotus izpildes ciklus un rada stabilu un prognozējamu slodzi emulatoram. Šī programma ir piemērota veiktspējas analīzei, jo ļauj novērtēt veiktspēju gan īstermiņā, gan ilgtermiņā.

Katrs tests tika veikts vairākas reizes, lai samazinātu sistēmas svārstību, piemēram, sistēmas fonu procesu ietekmi uz mērījumiem. Starp testiem emulators tika pilnībā apturēts un palaists no jauna, nodrošinot vienādus sākuma apstākļus. Testēšanas laikā netika mainīti emulatora iestatījumi vai kods, un netika palaistas citas programmas, kas varētu radīt slodzi uz sistēmu.

Vērts pieminēt, ka testēšana tika veikta uz vienas aparatūras konfigurācijas, tāpēc iegūtie rezultāti raksturo konkrēti iepriekšminētās aparatūras vidi. Rezultāti var atšķirties, izmantojot procesoru, operatīvo atmiņu vai operētājsistēmu ar citu specifikāciju, tomēr vispārējās tendences saglabājas.

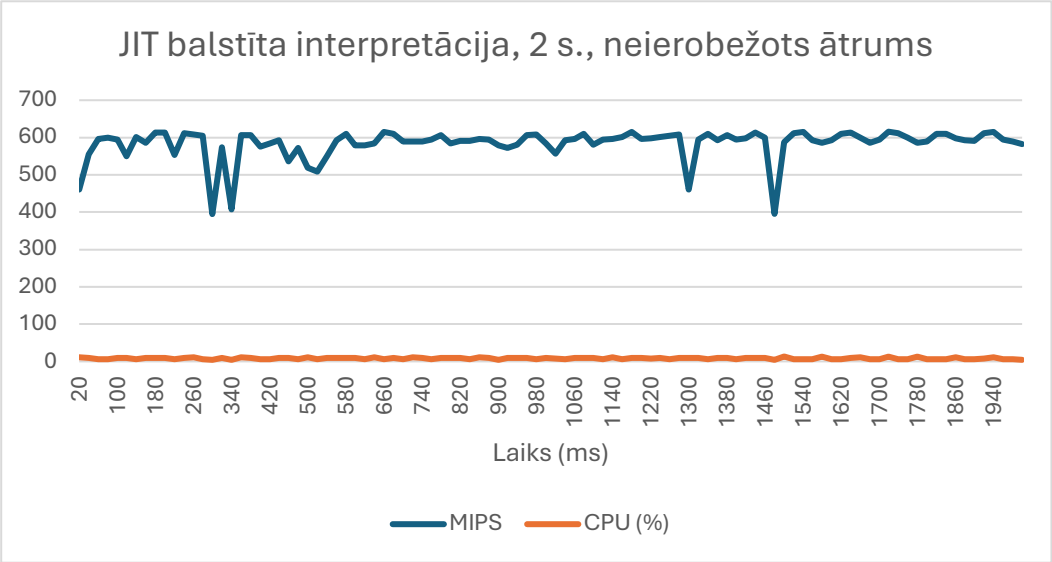
2.3. Rezultātu analīze

Veiktspējas mērījumos tika izmantoti kvantitatīvi rādītāji:

- Izpildes ātrums, kas tika mērīts un apzīmēts ar MIPS jeb miljonos instrukciju sekundē (angliski: Millions Instructions Per Second);
- Procesora noslodze, kas tika mērīta procentos un apzīmēta ar CPU.
- Operatīvās atmiņas noslodze, kas tika mērīta MB jeb megabaitos (angliski: megabytes) un apzīmēta ar RAM.

2.3.1. Īstermiņa veiktspējas analīze

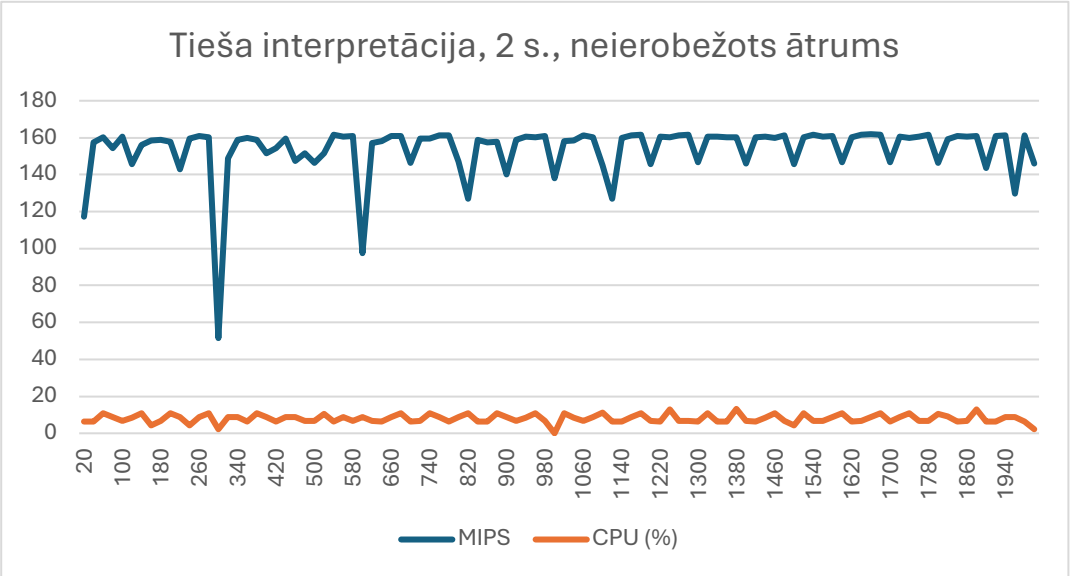
Īstermiņa testi tika veikti ar 2 sekunžu izpildes laiku testēšanas režīmā (bez izpildes ātruma ierobežojuma), kas īpaši labi ilustrē JIT balstītās interpretācijas sākotnējo uzvedību un pāreju uz stabilu darba režīmu. JIT balstītās interpretācijas testos būtiska nozīme bija kešatmiņas darbībai, kā arī mašīnkoda ģenerēšanai. Testa laikā mērījumi tika veikti katras 20 milisekundes.



(3. attēls: JIT balstītas interpretācijas mērījumi, īstermiņa tests)

JIT balstītās interpretācijas grafikā (skat. 3. att.) skaidri novērojama sākotnējā fāze, kuras laikā izpildes ātrums vēl nav pilnībā nostabilizējies. Pirmajos izpildes ciklos JIT kompilators identificē biežāk izmantotos koda blokus un veic to pārtulkošanu uz mašīnkodu. Šajā periodā MIPS vērtības var īslaicīgi samazināties, jo procesoram paralēli instrukciju izpildei ir jāveic arī kompilācijas un optimizācijas darbības.

Grafikā redzamie MIPS kritumi šajā kontekstā nav interpretējami kā veiktspējas pasliktināšanās, bet gan kā pārejas process uz optimizētu izpildes režīmu. Šie kritumi korelē ar brīžiem, kad tiek ģenerēts jauns mašīnkods un notiek kešatmiņas piepildīšana ar mašīnkodu. Kad ģenerētais mašīnkods kods ir saglabāts, turpmākā izpilde notiek ievērojami ātrāk un ar mazāku procesora slodzi.



(4. attēls: Tiešas interpretācijas mērījumi, īstermiņa tests)

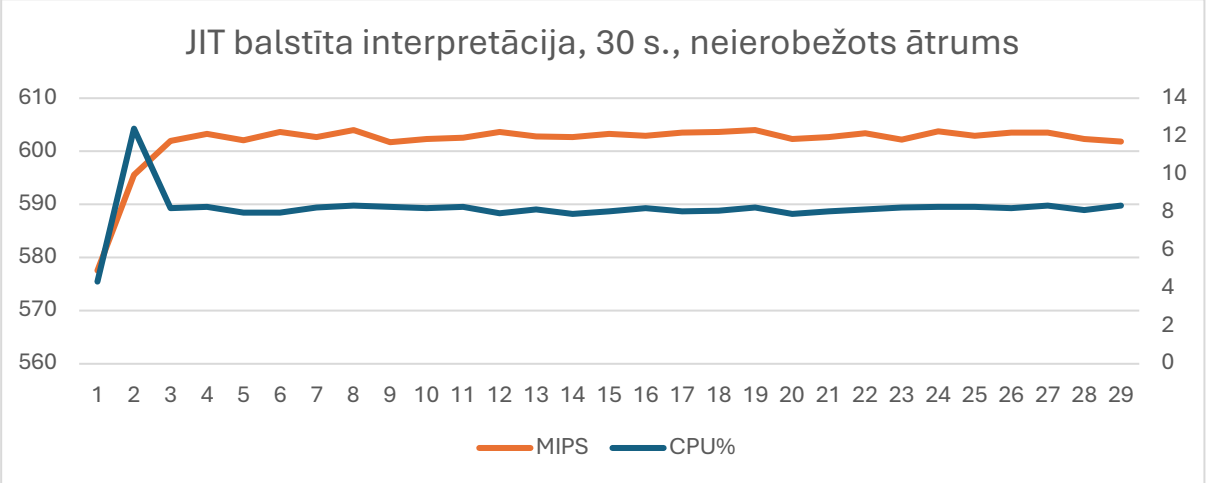
Tiešās interpretācijas gadījumā grafikā (skat. 4. att.) ir skaidri novērojamas MIPS svārstības un kritumi, kas parādās visas testēšanas laikā. Atšķirībā no JIT balstītās interpretācijas, šīs svārstības neveido pāreju uz stabilizētu izpildes režīmu, un tā ir pastāvīga parādība, kas raksturo tiešās interpretācijas mehānisma darbību.

Izpildes ātruma svārstības ir skaidrojamas ar to, ka katrā izpildes ciklā instrukcijas tiek atkārtoti nolasītas, dekodētas un izpildītas no jauna. Tā kā netiek ģenerēts atkārtoti lietojams mašīnkods, kešatmiņa netiek piepildīta ar biežāk izpildītajām instrukcijām, un līdz ar to sistēma nespēj sasniegt vienmērīgu veiktspējas fāzi.

Atšķirībā no JIT balstītas interpretācijas, kur MIPS kritumi galvenokārt koncentrējās mašīnkoda ģenerēšanas un saglabāšanas brīžos, tiešajā interpretācijā kritumi saglabājas visa testa garumā. Tas norāda uz strukturālu ierobežojumu interpretācijas pieeja, nevis uz īslaicīgu optimizācijas trūkumu. Rezultātā izpildes ātrums īstermiņā saglabājas zemāks un mazāk prognozējams. Šī patstāvīgā svārstību klātbūtne skaidri parāda, ka tiešā interpretācija nespēj pielāgoties izpildes raksturam. Tas būtiski atšķiras no JIT balstītās interpretācijas pieejas, kur sistēma laika gaitā uzkrāj optimizācijas ieguvumus un pakāpeniski samazina veiktspējas svārstības.

2.3.2. Ilgtermiņa veiktspējas analīze

Ilgtermiņa veiktspējas testi tika veikti ar 30 sekunžu izpildes laiku testēšanas režīmā, kas ļāva detalizēti apskatīt interpretatoru veiktspēju pēc sākotnējās izpildes fāzes, kad sistēma jau ir sasniegusi stabilu darba režīmu. Šādi testi ir īpaši nozīmīgi JIT balstītās interpretācijas gadījumā, jo tie parada, kā mašīnkoda ģenerēšana ietekmē veiktspēju ilgstošas darbības laikā. Mērījumi tika veikti katru sekundi.



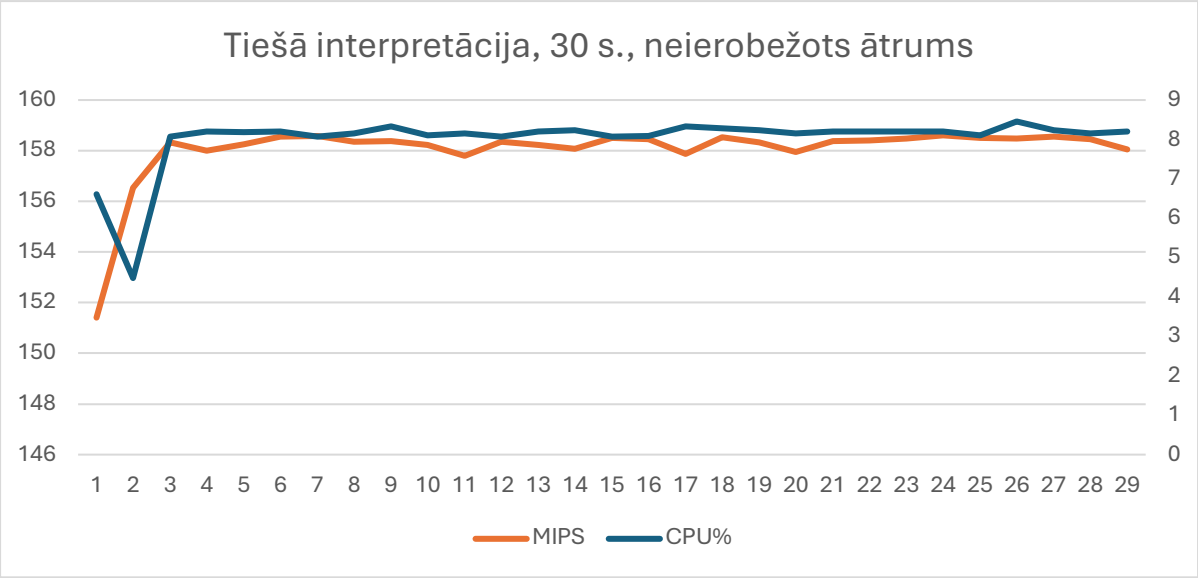
(5. attēls: JIT balstītas interpretācijas mērījumi, ilgtermiņa tests)

JIT balstītās interpretācijas ilgtermiņa veiktspējas grafikā (skat. 5. att.) ir novērojams, ka pēc sākotnējās kešatmiņas piepildīšanas fāzes izpildes ātrums stabilizējas. Par šo liecina MIPS kritums un procesora slodzes strauja paaugstināšanās, jo mašīnkoda ģenerēšana prasīja vairāk resursu. MIPS vērtības lielāko daļu testa laikā ieturējās 578-604 MIPS robežās, kas norāda uz to, ka biežāk izmantotās instrukcijas tika pārtulkotas uz mašīnkodu un veiksmīgi saglabātas kešatmiņā.

Atšķirībā no īstermiņa testiem, ilgtermiņa izpildē MIPS kritumi kļūst ievērojami retāki un mazāk izteikti. Tas liecina, ja JIT kompilatoram vairs nav nepieciešams bieži ģenerēt jaunu kodu, jo lielākā daļa instrukciju tika optimizēti. Šāda uzvedība atbilst JIT kompilācijas

teorētiskajam modelim, kur sākotnējās veiktspējas izmaksas tiek kompensētas augstāku veiktspēju ilgtermiņā.

Pēc sākotnējās fāzes procesora noslodze saglabājās vienmērīga, bez straujiem pieaugumiem vai kritumiem, kas norāda uz to, ka sistēma tajā brīdī sasniedza līdzsvarotu izpildes stāvokli, kurā procesors veica mašīnkoda izpildi, nevis interpretācijas vai kompilācijas darbības. Ilgtermiņā šāda noslodzes struktūra ir labvēlīga gan veiktspējas, gan energoefektivitātes ziņā.



(6. attēls: Tiešas interpretācijas mērījumi, ilgtermiņa tests)

Tiešās interpretācijas ilgtermiņa grafikā (skat. 6. att.) MIPS svārstības saglabājas visā testēšanas laikā. MIPS vērtības svārstās aptuveni 151-158 MIPS robežās, ar periodiskiem kritumiem, kas atkārtojas visā testa laikā un ir izteiktāki nekā JIT balstītās interpretācijas gadījumā.

Šāda uzvedība ir skaidrojama ar to pašu trūkumu, kas bija redzams arī īstermiņa testa rezultātos. Tiešajā interpretācijā katra instrukcija tiek izpildīta no jauna katrā ciklā, kešatmiņa netiek piepildīta un optimizācijas procesi nenotiek. Katrs izpildes cikls saglabā līdzīgu pieskaitāmo izmaksu līmeni, kas ierobežo maksimālo iespējamo veiktspēju.

Procesora noslodzes dati ir līdzīgi ar datiem no JIT balstītās interpretācijas ilgtermiņa testiem.

Interpretācijas pieeja	MIPS	CPU (%)	RAM (MB)
JIT	601,8054	8,167795	42,978531
Tiešā	158,0241	8,004929	42,162572

(1. tabula: Interpretācijas pieeju vidējie mērījumi ilgtermiņā)

Vidējās procesora noslodzes vērtības ilgtermiņa testa mērījumi (skat. 1. tabulu) rāda, ka JIT interpretācijas procesora patēriņš ir tikai nedaudz augstāks nekā tiešās interpretācijas gadījumā. Starpība starp abām pieejām ir aptuveni 0,17%, kas praksē ir uzskatāma par nenozīmīgu. Šāds rezultāts norāda, ka dinamiskā koda pārtulkošana un optimizācija JIT balstītajā interpretācijā nerada būtisku papildu slodzi procesoram stabilā darba fāzē.

Nelielais procesora noslodzes pieaugums JIT režīmā skaidrojams ar fona uzturēšanas darbībām, piemēram, optimizētā koda pārvaldību, atmiņas adresācijas mehānismiem un

iespējamu kešatmiņas invalidāciju, taču šīs izmaksas ir minimālas un būtiski neietekmē sistēmas kopējo veiktspēju.

Svarīgi uzsvērt, ka, neskatoties uz gandrīz identisku procesora noslodzi, JIT interpretācija nodrošina būtiski augstāku izpildes ātrumu. Salīdzinot tiešās interpretācijas un JIT balstītās interpretācijas vidējās MIPS vērtības, var secināt to, ka JIT balstīta interpretācija ir aptuveni 3,8 reizes ātrāka, patērējot praktiski tādu pašu procesora un operatīvās atmiņas resursu apjomu. Tas nozīmē, ka JIT balstīta interpretācija nodrošina ievērojami labāku veiktspēju uz vienu CPU procentpunktu, kas ir būtisks rādītājs sistēmu efektivitātes novērtēšanā.

Šādi rezultāti skaidri parāda, ka JIT interpretācijas priekšrocības izpaužas nevis kā lielāks resursu patēriņš, bet gan kā efektīvāka šo resursu izmantošana, par to liecina tas, ka tiešā interpretācija patērē līdzīgu procesora resursu apjomu, taču sasniedz zemāku izpildes ātrumu, jo tas katrā ciklā atkārtoti veic instrukciju dekodēšanu, kas neļauj sasniegt augstu instrukciju izpildes blīvumu.

2.3.3. Rezultātu interpretācija

Šajā nodaļā iegūtie rezultāti tika interpretēti balstoties uz teorētiskajā daļā apskatītajām interpretācijas pieejām, lai novērtētu, cik lielā mērā eksperimentālie dati apstiprina iepriekš izvirzītos pieņēmumus par tiešās un JIT balstītās interpretācijas darbības principiem.

Teorētiskajā daļā tika secināts, ka tiešā interpretācija katru instrukciju apstrādā neatkarīgi, atkārtoti veicot dekodēšanu un izpildi, kas rada papildu pieskaitāmās izmaksas katrā ciklā, turpretī JIT balstīta interpretācija paredz dinamisku biežāk izmantoto izpildāmo instrukciju pārtulkošanu uz mašīnkodu, kas ilgtermiņā paliela veiktspēju.

Praktiskajā daļā iegūtie mērījumi pilnībā apstiprina šos teorētiskos pieņēmumus. JIT balstītās interpretācijas gadījumā sākotnējā izpildes fāzē bija novērojami īslaicīgi veiktspējas kritumi, kas saistīti ar mašīnkoda ģenerēšanu un kešatmiņas piepildīšanu, taču pēc šīs pārejas fāzes emulators sasniedza stabilu darba režīmu, kurā izpildes ātrums ievērojami pārsniedza tiešās interpretācijas rezultātus.

Ilgtermiņa testos JIT balstīta interpretācija sasniedza aptuveni 578-604 MIPS, kamēr tiešā interpretācija saglabājās 151-159 MIPS robežās. Tas nozīmē, ka JIT balstītā pieeja konkrētajā CHIP-8 emulatora realizācijā ir aptuveni 3,8 reizes ātrāka. Šī atšķirība ir būtiska un skaidri demonstrē koda dinamiskās pārtulkošanas efektivitāti pat ļoti vienkāršas arhitektūras gadījumā.

Svarīgi ir arī tas, ka, neskatoties uz būtiski lielāku izpildes ātrumu, procesora noslodze JIT balstītas interpretācijas gadījumā bija tikai nedaudz augstāka nekā tiešas interpretācijas gadījumā. Procesora noslodzes starpība bija mazāka par vienu procentu, kas praksē ir uzskatāma par nenozīmīgu. Tas norāda, ka JIT balstīta interpretācija nepalielina resursu patēriņu proporcionāli veiktspējas pieaugumam, bet gan efektīvāk izmanto pieejamos sistēmas resursus.

Operatīvās atmiņas patēriņš abās pieejās bija ļoti līdzīgs. Lai gan JIT balstītai interpretācijai būtu jāpatērē vairāk atmiņas mašīnkoda glabāšanai, praktiskajā realizācijā šis pieaugums bija minimāls, šo varētu skaidrot ar to, ka CHIP-8 instrukciju kopa ir maza un pārtulkotā koda apjoms ir salīdzinoši neliels.

Kopumā iegūtie rezultāti pilnībā saskan ar teorētiskajā daļā aprakstītajiem interpretācijas pieeju darbības principiem. Dati pierāda, ka JIT balstīta pieeja ir īpaši efektīva scenārijos, kuros viena un tā pati instrukciju secība tiek izpildīta atkārtoti, kā tas ir raksturīgs emulatoriem, un ka

JIT balstītas interpretācijas ieguvumi kļūst pamanāmi ilgtermiņa izpildē, kamēr programmām ar īslaicīgu izpildi tiešā interpretācija var būt pietiekama.

2.4. Testa vides ierobežojumi un kļūdas

Veicot jebkuru veiktspējas analīzi un eksperimentālu pētījumu, ir būtiski identificēt iespējamos faktorus, kas var ietekmēt iegūto rezultātu precizitāti. Šajā pētījumā, neskatoties uz kontrolētiem testēšanas apstākļiem, pastāv vairāki aspekti, kas jāņem vērā, interpretējot rezultātus.

Viens no galvenajiem ierobežojumiem ir tas, ka testēšana tika veikta tikai uz vienas aparatūras konfigurācijas. Procesora arhitektūra, kešatmiņas lielums, instrukciju kopas īpatnības un operētājsistēma var būtiski ietekmēt interpretatora efektivitāti. Piemēram, procesori ar lielāku kešatmiņu un modernāku zaru prognozēšanu var dot vēl labākus rezultātus JIT balstītas interpretācijas testos, savukārt vecākās sistēmās veiktspējas pieaugums var būt mazāk izteikts.

Otrs būtisks faktors ir izmantoto testa programmu skaits un raksturs. Šajā darbā tika izmantota viena intensīva testa programma, kas atkārtoti izpilda līdzīgas instrukciju secības. Šāds scenārijs ir īpaši labvēlīgs JIT balstītajai interpretācijai, jo tas ļauj izmantot dinamisko koda pārtulkošanu, taču programmās ar mazāku atkārtojamību vai ar lielu skaitu reti izpildāmu zaru JIT balstītas interpretācijas priekšrocības var būt mazāk izteiktas.

Jāņem arī vērā arī tas, ka emulators tika izstrādāts ar mērķi salīdzināt interpretācijas pieejas, nevis maksimāli optimizēt katru realizācijas detaļu. Tiešās interpretācijas gadījumā netika ieviestas papildu optimizācijas, piemēram, instrukciju iepriekšēja dekodēšana, kas teorētiski varētu uzlabot tās veiktspēju, līdz ar to rezultāti atspoguļo salīdzinājumu starp tiešās interpretācijas un JIT balstītas interpretācijas pamata modeļiem, nevis to optimizētiem risinājumiem.

Papildu kļūdu avots var būt operētājsistēmas fona procesi un resursu plānošana. Lai gan testēšanas laikā tika samazināta citu programmu ietekme, pilnībā izslēgt ārējo procesu ietekmi nav iespējams. Šie procesi var izraisīt nelielas svārstības procesora noslodzes un izpildes ātruma mērījumos, īpaši īstermiņa testos.

Neskatoties uz šiem ierobežojumiem, darba rezultāti ir pietiekami atkārtojami, lai izvirzītu pamatotus secinājumus par tiešas un JIT balstītas interpretācijas atšķirībām CHIP-8 emulatora kontekstā. Testa vides ierobežojumu un iespējamu kļūdu avoti nevis mazina darba vērtību, bet palīdz interpretēt rezultātus un norāda virzienus turpmākiem pētījumiem.

Secinājumi

Šī zinātniski pētnieciskā darba mērķis bija salīdzināt tiešo interpretācijas metodi ar JIT balstītu interpretāciju, izmantojot CHIP-8 emulatoru kā pētījuma platformu. Darba gaitā tika izstrādāti divi emulatora izpildes režīmi, veikti veiktspējas mērījumi un analizēti iegūtie dati.

Darba sākumā izvirzītā hipotēze paredzēja, ka JIT balstīta interpretācija nodrošinās lielāku izpildes ātrumu, savukārt tiešā interpretācija patērēs mazāk sistēmas resursu. Praktiskās daļas rezultāti parādīja, ka hipotēze ir daļēji apstiprinājusies. JIT interpretācija patiešām nodrošināja ievērojami augstāku izpildes ātrumu, taču resursu patēriņa ziņā atšķirības starp abām metodēm bija minimālas.

Veiktspējas mērījumi skaidri parādīja, ka JIT balstītā interpretācija ilgtermiņa izpildē ir vairākas reizes ātrāka nekā tiešā interpretācija. Šis veiktspējas pieaugums ir īpaši nozīmīgs

emulatoru kontekstā, kur viena un tā pati instrukciju secība bieži tiek izpildīta tūkstošiem vai pat miljoniem reižu. JIT interpretācija šādos apstākļos spēj efektīvi samazināt sākotnējās koda pārtulkošanas izmaksas.

Tajā pašā laikā tika konstatēts, ka procesora un operatīvās atmiņas noslodze abos izpildes pieejās ir ļoti līdzīga. Tas liecina, ka JIT balstītas interpretācijas priekšrocība nav saistīta ar lielāku resursu patēriņu, bet gan ar efektīvāku šo resursu izmantošanu. Šis secinājums ir būtisks, jo tas parāda, ka JIT balstītu interpretāciju var izmantot arī sistēmās ar ierobežotiem resursiem, ja tiek nodrošināta pietiekama izpildes ilguma atkārtojamība.

Darba ierobežojums ir tas, ka testēšana tika veikta tikai uz vienas aparatūras konfigurācijas un izmantojot vienu galveno testa programmu. Rezultāti var atšķirties, ja emulators tiek palaists uz citas arhitektūras vai ja tiek izmantotas programmas ar atšķirīgu instrukciju izpildes raksturu. Tomēr iegūtās tendences ir vispārīgas un atbilst teorētiskajiem modeļiem, kas apraksta JIT balstītas interpretācijas darbību.

Noslēgumā var secināt, ka JIT balstīta interpretācija ir efektīva pieeja emulatoru izstrādē, īpaši tādos emulatoros, kur prioritāte ir augsta veiktspēja un ilgtermiņa izpilde. Tiešā interpretācija saglabā savu nozīmi vienkāršos vai īslaicīgos scenārijos, taču modernās sistēmās JIT balstīta pieeja piedāvā ievērojamas priekšrocības.

Izmantotās literatūras saraksts

1. Thomas P. Greene. (1997). Cowgod's Chip-8 Technical Reference v1.0. Frédéric Devernay. Skatīts: 22.11.2025. Pieejams: http://devernay.free.fr/hacks/chip8/C8TECH10.HTM
2. Tobias V. Langhoff. (2020). Guide to making a CHIP-8 emulator. Skatīts: 22.11.2025. Pieejams: https://tobiasvl.github.io/blog/write-a-chip-8-emulator/
3. John Aycock. (2003). A Brief History of Just-In-Time. University of Calgary. Skatīts: 23.11.2025. Pieejams: https://www.cs.tufts.edu/comp/150IPL/papers/aycock03jit.pdf
4. Keith D. Cooper, Linda Torczon. (2012). Engineering a compiler.
5. Pheonixnap.de. (2024). Was ist Emulation? Skatīts: 24.11.2025. Pieejams: https://phoenixnap.de/Glossar/was-ist-emulation
6. Vinicius Fulber-Garcia, Michal Aibin. (2024). Differences Between Simulation and Emulation. Baeldung.com. Skatīts: 04.12.2025. https://www.baeldung.com/cs/simulation-vs-emulation
7. Austin Morlan. (2019). Building a CHIP-8 Emulator. Pieejams: https://austinmorlan.com/posts/chip8_emulator/
8. David A. Patterson, John L. Hennessy. (2020). Computer Organization and Design: The Hardware/Software Interface. (6th Edition).
9. Eric Kaltman, Winnie Schwaid-Lindner, Drey Jonathan, Andrew Borman, Alex Garnett, Larry Masinter. (2025). An Overview of Emulation as a Preservation Method. Skatīts: 05.12.2025. Pieejams: https://www.clir.org/wp-content/uploads/sites/6/2025/07/An_Overview_of_Emulation_as_a_Preservation_Method_CLIRpub194.pdf

Pielikums

1. Github, autora veidotie CHIP-8 emulatori. Pieejams: https://github.com/4soobs/chip8emu
2. Libretro, CHIP-8 programmu arhīvs. Pieejams: https://buildbot.libretro.com/assets/cores/CHIP-8/
3. Lietotā saimniekdatora specifikācija:

Komponente	Specifikācija
Procesors (CPU)	AMD Ryzen 5 5600G
Operatīvā atmiņa	2x8GB (kopā 16GB) DDR5
Operētājsistēma	Windows 11 Pro 25H2